Phase 5 explained — Feature engineering

Merged NetFlow data · notebook `08_features_netflow.ipynb`

1. What is feature engineering?

A model does not read network traffic; it reads a table of numbers, one column per **feature**. Feature engineering prepares that table so that (1) each column means something about the traffic, (2) the columns are in a form a model can use, and (3) useless or misleading columns are removed. The requirement asks for four techniques and one justification:

> **Requirement:** explain feature **selection**, **extraction/creation**, **encoding**, and **scaling/transformation** when appropriate; **justify the final feature set**.

No train/test split. The project does not train a model, so the data are not split. In a real modelling phase, the steps that "learn" from the data (scaling, selection) would be fitted on the training part only, to avoid *data leakage* (test information sneaking into training).

2. The steps in one table

Step	What we did	Result
0. Starting columns	Removed columns that reveal the answer: timestamps, IPs, TTL, random IDs	31 numeric + 8 code columns
1. Creation	Built 9 new columns from existing ones	ratios, rates, 4 yes/no behaviour flags
2. Encoding	Turned code columns into yes/no columns	32 new 0/1 columns
3. Transformation	$\log(1+x)$ on skewed numbers	strongly skewed columns: 36 → 19
3. Scaling	$(x - \text{median})/\text{IQR}$	all numbers on a similar scale
4. Selection	Removed fingerprints, near-constant and duplicate columns	72 candidates → **59 features**
5. Final set	Table of every feature + saved dataset	`netflow_features.parquet`
6. PCA	2-D map of the families	families overlap: many features needed

3. Each step, simply

Step 0 — Starting columns. Some columns would let a model "cheat": a timestamp tells *when* the lab ran an attack, an IP tells *which machine* attacked, and TTL identifies the lab's computers. None of them describes the traffic itself, so they are removed (phase 2 decisions M2–M5, M10).

Step 1 — Creation (extraction). New columns, computed row by row:

- **Ratios:** bytes per packet (sent and received), and bytes received per byte sent. They describe the *shape* of a flow, not its size. In phase 4 the raw sizes pointed in opposite directions in different networks.
- **Rates:** packets per second, bytes per second. Floods are fast; slow attacks are slow.
- **Behaviour flags (0/1):** `no_answer` (the server never replied), `one_packet`, `syn_only` (a half-open scan), `hit_timeout` (the flow lasted 120 s or more).

Example: 92% of Reconnaissance flows are one-packet, SYN-only probes. One column now says "this is a scan".

Step 2 — Encoding. Some columns are **codes, not quantities**: port 443 is not "more" than port 80.

- **One-hot encoding:** one 0/1 column per category, e.g. `proto_tcp`, `proto_udp`, `proto_other`.
- **Port:** 65,536 values, so they are grouped into 3 official ranges: well-known (0–1023), registered, dynamic.
- **TCP flags:** the value 27 is a bitmask meaning SYN + ACK + PSH + FIN, so it becomes one 0/1 column per flag.
- **Application protocol:** the 9 protocols above 1% each get a column; the rare ones are grouped as "other".

Step 3 — Transformation and scaling.

- **Why transform?** The numbers are *extremely skewed*: most flows are small, a few are gigantic (skewness up to 319). $\log(1+x)$ shrinks big values (each $\times 10$ becomes $+2.3$), keeps 0 at 0, and keeps the order.
- **Why scale?** After the log, columns still have different ranges. **Robust scaling** computes $(x - \text{median})/\text{IQR}$, where the IQR is the width of the middle 50% of values.
- **Why "robust"?** We use the median and IQR, not the mean and standard deviation, because the extreme values *are* the attacks and must not set the scale.
- **Special case:** if more than half of the values are equal (e.g. retransmissions are 0 for most flows), the IQR is 0. Those 5 columns are divided by the standard deviation instead; otherwise they would dominate.

Step 4 — Selection. Each removal has a reason:

1. **Exact copies:** none were left after phase 3.
2. **Fingerprints:** the TCP window size, which is set by each lab machine's operating system. A model would learn "which lab", not "which attack".
3. **Near-constant columns:** the same value in $\geq 99.5\%$ of rows carries no information (3 rare TCP flags).
4. **Correlated pairs:** if two columns move together ($|\rho| \geq 0.95$, Spearman correlation), they say the same thing, so one is removed (8 removed). Example: retransmitted packets vs retransmitted bytes.
5. **Mutual information (MI):** a score of how much a feature tells about the attack family (0 = nothing). It decides which column of a pair to keep, and ranks the final features.

Step 5 — Final feature set (the justification). 59 features: 25 original, 8 created, 26 encoded. Why this set is good:

- **every feature is justified** by phase 2 or phase 4;
- **nothing identifies the lab, the machine or the moment;**
- **no duplicates and no useless columns;**
- **everything is ready for a model** (scaled and encoded).

Proof that creation was worth it: 3 of the 5 most informative features are created ones (bytes per packet sent and received, and the out/in ratio), and 8 of the 9 created features survived the selection.

Step 6 — PCA. Principal component analysis squeezes the 59 features into 2 new axes so we can draw the data on a map. Two axes keep 42.8% of the information. Reconnaissance and Fuzzing form their own clusters, but most families overlap. So, as phase 4 already showed, **no single direction separates the attacks**: a model needs many features together.

Step 7 — Techniques not used ("when appropriate"). Each was considered and rejected for a reason:

- **hour of day:** it leaks the lab's attack schedule;
- **polynomial terms:** no curved relationships, and they would add about 1,800 columns;
- **binning of sizes:** the log keeps more detail;
- **target encoding:** it uses the label, so there's a risk of leakage;
- **train/test split:** there is no model.

4. Six words to know

Feature	one input column of the model
One-hot encoding	one 0/1 column per category of a code column
Skewness	how lopsided a distribution is: 0 = symmetric, above 1 = strongly skewed
IQR	Q3 − Q1, the width of the middle half of the values; not affected by extremes
Mutual information	how much knowing a feature reduces the uncertainty about the family (0 = no help)
Data leakage	information from the test data used while preparing or training, which makes results look too good

5. The answer in three sentences

We built 9 behaviour features, encoded the code columns, log-transformed and robust-scaled the numbers, and removed fingerprints, constant and duplicate columns, ending with **59 justified features**. The created features are among the most informative, which confirms the phase 4 findings. The set contains only traffic behaviour, nothing that identifies the lab, so a future model has to learn the attack, not the network (SQ3).